Structures

Mahir Labib Dihan
Lecturer, CSE, BUET

User-Defined Types: Structures

Why Do We Need Structures?

Problem: How do we store information about a student?

Without structures:

- `char name[50];`
- `int roll;`
- `float cgpa;`
- `int age;`

Separate variables — hard to manage!

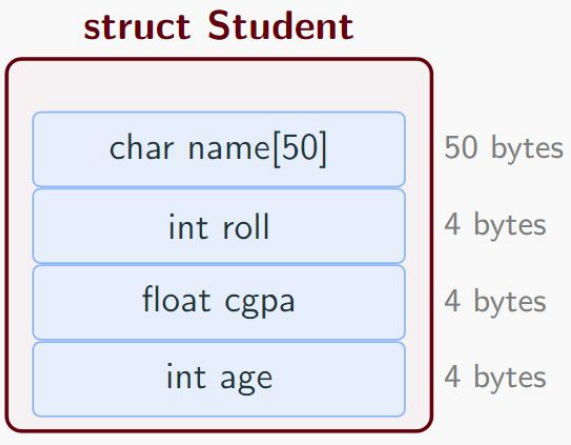
With structures:

- Group all related data together
- One variable holds everything
- Easy to pass to functions
- Can create arrays of students

What is a Structure?

Definition

A **structure** is a user-defined data type that groups related variables of different types under a single name.



Declaring a Structure

```c
// Structure definition
struct Student {
      char name[50];
      int roll;
      float cgpa;
      int age;
}; // Don't forget the semicolon!

int main() {
      // Declare a variable of type struct Student
      struct Student s1;

      return 0;
}
```

Initializing Structures

```
// Method 1: Initialize at declaration
struct Student s1 = {"Alice", 101, 3.85, 20};

// Method 2: Initialize members individually
struct Student s2;
strcpy (s2.name, "Bob"); // For strings, use strcpy
s2.roll = 102;
s2.cgpa = 3.72;
s2.age = 21;

// Method 3: Designated initializers
struct Student s3 = {.roll = 103, .name = "Charlie", .cgpa = 3.90};
```

Accessing Structure Members

Use the **dot operator (.)** to access members:

```
struct Student s1 = {"Alice", 101, 3.85, 20};

// Reading members
printf("Name: %s\n", s1.name);
printf("Roll: %d\n", s1.roll);
printf("CGPA: %.2 f\n", s1.cgpa);

// Modifying members
s1.age = 21;
s1.cgpa = 3.90;
```

Output:

```
Name: Alice
Roll: 101
CGPA: 3.85
```

Array of Structures

Store multiple records using an array:

```
struct Student class[100]; // Array of 100 students

// Initialize first student
strcpy(class[0].name, "Alice");
class[0].roll = 101;
class[0].cgpa = 3.85;

// Loop through all students
for (int i = 0; i < 100; i++) {
     printf("Student %d: %s (Roll : %d)\n",
          i+1, class[i].name, class[i].roll);
}
```

Structures and Functions

```c
// Function that takes a structure as parameter
void printStudent(struct Student s) {
      printf("Name : %s, Roll: %d, CGPA: %.2f\n", s.name, s.roll, s.cgpa);
}

// Function that returns a structure
struct Student createStudent(char *name, int roll, float cgpa) {
      struct Student s;
      strcpy(s.name, name);
      s.roll = roll;
      s.cgpa = cgpa;
      return s;
}

int main() {
      struct Student s = createStudent("Alice", 101, 3.85);
      printStudent(s);
      return 0;
}
```

typedef: Simplifying Structure Usage

```c
// Instead of writing "struct Student" everywhere...
typedef struct {
      char name[50];
      int roll;
      float cgpa;
} Student; // Now we can just use "Student"

int main() {
      Student s1; // No need for "struct" keyword
      Student class[100];

      s1.roll = 101;
      strcpy(s1.name, "Alice");

      return 0;
}
```

typedef creates an **alias** for the structure type.

Nested Structures

```
typedef struct {
      int day, month, year;
} Date;

typedef struct {
      char name[50];
      int roll;
      Date birthdate; // Nested structure
      Date admission; // Another nested structure
} Student ;

int main() {
      Student s;
      s.birthdate.day = 15;
      s.birthdate.month = 8;
      s.birthdate.year = 2003;

      printf("Born: %d/%d/%d\n", s.birthdate.day, s.birthdate.month, s.birthdate.year);
      return 0;
}
```

Summary

Summary

Structures:

- Group related data
- Access with dot operator
- Use `typedef` for convenience
- Can nest and create arrays

Acknowledgement

- Mohammad Sadat Hossain, Lecturer, CSE, BUET